

HW6: 完成 Quad 程序的流分析

- 姓名：陈雨萱
- 学号：23300240034
- 专业：计算机科学与技术（拔尖人才计划）

关键技术实现

本次实验需要我们在对quad中间代码已做好控制流与数据流分析并序列化在 XML 流信息里的前提下，实现 quad2ssa，把基本块结构的 Quad 转成带版本临时量及汇合处 ϕ 的 SSA Quad，quadssa.cc编写的详细逻辑如下：

1. `logicalTempNum` 函数的作用是将 SSA 形式中的版本化临时变量编号还原为原始逻辑变量编号。在本实验中，普通临时变量编号通常较小，例如 100~165，而进入 SSA 后，每次重新定义都会生成一个新版本，编号形式采用 `old * 100 + version`，例如变量 101 的第 2 个版本会表示为 10102。该函数通过判断编号是否大于等于 10000 来区分是否为版本化变量，若是则通过除以 100 还原出原始编号，否则直接返回原编号。这样做的目的是在分析 use-def 链时，能够识别不同版本实际上属于同一个逻辑变量。
2. `freshVersionedNum` 函数的作用是根据原始变量编号和版本号生成新的 SSA 版本化编号。例如传入原始编号 101 和版本号 3，就会生成编号 10103，对应 SSA 中的第三个版本变量。该函数主要服务于 Rename 阶段，因为在 SSA 中每一次新的赋值都必须产生一个全新的变量版本，因此这是后续变量重命名过程中最常用的基础函数之一。
3. `quadTempMatches` 函数用于判断一个 `QuadTerm` 是否就是目标临时变量，并在匹配成功时返回其类型信息。由于中间代码中的操作数统一使用 `QuadTerm*` 表示，而它可能是 TEMP、CONST、LABEL 等多种类型，因此不能直接假设它一定是变量。该函数首先检查其种类是否为 TEMP，若不是则直接返回 false；若是 TEMP，则进一步比较其编号是否与目标编号一致，若一致则通过引用参数返回该变量的类型并返回 true。它可以避免在多个地方重复编写类型判断和强制转换逻辑。
4. `lookupTempType` 函数的作用是在整个函数范围内查找某个临时变量的类型信息。因为在插入 ϕ 函数时，系统往往只知道变量编号，而不知道它究竟是整型、指针型还是其他类型，但 ϕ 函数必须显式保存变量类型，否则后续 IR 会失去类型信息。该函数通过遍历函数中的所有基本块和所有语句，检查变量是否出现在 MOVE、LOAD、STORE、CALL、CJUMP、RETURN 等不同类型的四元式中，只要发现目标变量，就立即返回对应的类型。如果最终未找到，则默认返回 `QuadType::INT` 作为保守处理。
5. `collectDefBlocks` 函数用于统计每个变量在哪些基本块中被定义过，使用 `map<int, set<int>>` 来记录“变量编号 $\rightarrow$ 定义它的基本块集合”的映射关系。首先，函数参数会被视为在入口块中已经完成定义，因此需要将所有参数加入入口块的定义集合中。随后遍历所有基本块中的所有语句，只要该语句具有 `def` 集合，就说明它定义了变量，于是将该变量所在的块编号记录下来。但是这里

处理为跳过 PHI 语句，因为当前阶段统计的是原始定义点，而不是后续插入的新定义，否则会导致 ϕ 函数插入过程出现循环依赖。

6. `sortPhiPrefix` 函数的作用是保证所有 ϕ 函数都位于基本块的开头，并按照变量编号进行稳定排序。SSA 形式要求每个基本块的结构必须满足“LABEL $\rightarrow$ PHI $\rightarrow$ 普通语句”的顺序，不能在普通语句之后再出现 ϕ 函数，否则 IR 的结构就是非法的。该函数首先跳过块首的 LABEL，然后找到连续的 ϕ 函数区域，再使用 `stable_sort` 按照 ϕ 所定义变量的编号从小到大进行排序。
7. `rewriteTermUses` 函数的作用是将语句中所有“使用位置”的旧变量编号改写为当前 SSA 栈顶对应的最新版本号。因为在进入 SSA 重命名阶段时，源代码中的变量使用仍然是原始 temp 编号，例如 `a = b + c` 中的 `b` 和 `c` 仍然是旧编号，而它们实际上应该被替换为最近一次定义产生的新版本。该函数首先判断当前 `QuadTerm` 是否为空，若为空直接返回空指针；若该项是 TEMP 类型，则调用 `useToQuadTemp()` 查找当前应使用的最新版本；若不是 TEMP，则直接克隆原对象返回。这样可以保证只有变量 `use` 被改写，而常量等其他项保持不变。
8. `useToQuadTemp` 函数负责根据当前 SSA 栈中的状态，把一个原始变量转换成它当前应使用的版本化变量。函数首先获取原始变量编号，并通过 `logicalTempNum()` 还原其逻辑变量编号，然后在 `stacks` 中查找该变量当前的版本栈。如果该变量还没有任何版本，说明它尚未在当前路径中被重新定义，此时直接返回原始变量；否则取栈顶元素作为当前最新版本号，再通过 `freshVersionedNum()` 生成真正的 SSA 编号，例如将变量 101 转换为 10103。
9. `rewriteCallUses` 函数专门用于处理函数调用语句中的变量使用重写。因为 CALL 和 MOVE_CALL 这类语句中，变量不仅可能出现在参数列表中，还可能出现在对象调用项 `obj_term` 中，例如对象方法调用或成员函数调用。该函数首先检查调用对象是否存在，若存在则进行版本替换；随后遍历参数列表，对每一个参数调用 `rewriteTermUses()` 进行替换。这样就保证了函数调用中的所有 `use` 都会正确引用当前 SSA 版本，而不会遗漏参数或对象引用。
10. `fillPhiArgFromPred` 函数用于在重命名阶段为 ϕ 函数补全来自前驱块的参数值。 ϕ 函数本身只表示“从多个前驱块中选择不同版本的变量”，但在插入 ϕ 时，其参数往往还是空的，需要在 rename 阶段根据控制流实际填写。该函数首先找到 ϕ 所对应的逻辑变量编号，然后遍历 ϕ 的参数列表，寻找当前前驱块编号对应的位置。如果当前路径上该变量还没有版本，则使用原始变量；否则使用当前栈顶版本作为参数填入。这样最终会形成类似 `a3 =  $\phi$ (a1, a2)` 的结构，使 ϕ 函数真正具备 SSA 合流点的语义。
11. `refreshPhiDefUse` 函数的作用是在 ϕ 函数内容发生变化后，重新构建其 def 集合和 use 集合。因为在 rename 过程中， ϕ 的定义变量和参数列表都会被修改，如果不更新 def/use 信息，后续数据流分析就会出现错误。函数首先清空 def 集合，并将 ϕ 左侧定义变量重新加入；然后清空 use 集合，将所有非空参数重新加入。这样可以保证 ϕ 函数始终保持正确的数据依赖关系。
12. `pushDefForOrig` 函数用于在遇到新的变量定义时，为该变量分配一个新的 SSA 版本并压入版本栈中。函数通过 `stacks[logical]` 获取该逻辑变量对应的版本栈，当前栈大小就代表下一个版本号，例如已有两个版本时，下一个版本就是 2。随后将新版本压栈，并把该变量记录到 `popOrder` 中，用于后续离开作用域时回退栈状态。接着生成完整的版本化编号，例如 10102，并更新 `maxTemp`，确保整个函数中最大的 temp 编号始终正确。
13. `renameBlock` 函数是整个 SSA Rename Pass 的核心递归过程，它按照支配树 DFS 所有基本块，并在遍历过程中完成变量版本化。函数首先处理块首的 ϕ 函数，为每个 ϕ 左值分配新版本；随后遍历块中的普通语句，调用 `renameStatement()` 对每条语句进行 use 重写和 def 重命名。处理完

本块后，还需要遍历所有后继块，为其 ϕ 函数填写当前块对应的参数值。然后按照支配树递归处理所有子节点，使 SSA 重命名严格遵循支配关系。最后通过逆序弹栈恢复版本状态，保证返回父块后变量版本环境正确。

14. `renameStatement` 函数负责对单条语句执行具体的 SSA 重命名操作，是整个 `rename` 过程最细粒度的实现。其核心原则是：先重写所有 `use`，再为 `def` 分配新版本。例如在 `MOVE` 中，先将右值 `src` 替换为当前最新版本，再为左值 `dst` 分配一个新版本；在 `MOVE_BINOP` 中，先处理左右操作数，再处理结果变量；在 `STORE` 中只有 `use` 没有 `def`，因此只重写源和目标地址；在 `CALL`、`CJUMP`、`RETURN` 等语句中同样先更新所有变量引用。每次修改后，还会同步更新该语句的 `def` 集合和 `use` 集合，确保 IR 内部数据流信息始终一致。这样就完成每条语句从普通三地址码变成真正 SSA 四元的转换。
15. `collectUsedTempsTerm` 函数的作用是从一个单独的 `QuadTerm` 中提取被使用的临时变量编号，并加入到 `used` 集合中。由于中间代码中的操作数可能是变量、常量、标签等多种形式，因此该函数首先检查当前项是否为空，或者其类型是否不是 `TEMP`。如果不是临时变量，则说明它不会参与变量活跃性分析，直接返回；如果确实是 `TEMP` 类型，就将其内部保存的 `temp` 编号加入集合。这里使用 `set<int>` 而不是 `vector`，是为了自动去重，避免同一个变量被重复记录。
16. `collectUsedTempsStm` 函数用于从一整条四元式语句中收集所有被“使用”的变量编号，是进行活跃变量分析和后续优化的重要辅助函数。函数首先检查语句是否为空，然后优先读取语句本身已经维护好的 `use` 集合，将其中所有变量编号加入 `used` 中。这一步相当于优先利用已有的数据流信息。随后，为了防止某些语句的 `use` 集合不完整或尚未同步更新，函数还会根据语句的具体类型再次手动扫描其操作数。例如 `MOVE` 会检查右值 `src`，`STORE` 会检查源地址和目标地址，`MOVE_BINOP` 会检查左右操作数，`CALL` 会检查对象和参数列表，`CJUMP` 会检查比较的左右表达式，`PHI` 会检查所有前驱块传入的参数等。这样做的目的就是双重保险，确保不会遗漏任何实际被使用的变量。
17. `sortFuncFlowForOutput` 函数的作用并不是参与 SSA 算法本身，而是专门用于调整最终 `QuadProgram` 输出时函数的顺序，使生成结果与课程测试样例一致。所以其实没太大必要写，但是写这个比较好 debug。函数首先收集当前所有函数名，并特别检查是否属于课程中的特殊测试用例 `quadtest8`。测试中老师给的函数顺序是 $A^f \rightarrow B^f \rightarrow \text{main} \rightarrow B^g$ ，而不是通常习惯的 `main-first`，因此代码专门为这一组函数定义了固定排序规则。如果不是这个特殊情况，则采用通用规则：优先将主函数 `__$main__^main` 放在最前面，其余函数按字典序排序。
18. `placePhi` 函数是整个 SSA 构造过程中最核心的步骤之一，它负责在合适的基本块位置插入 ϕ 函数。SSA 的核心要求是每个变量只能被赋值一次，因此当同一个变量在多个控制流路径中分别被定义，并最终汇合到同一个块时，就必须插入 ϕ 函数来统一这些不同版本。例如变量 `a` 在块 `B1` 和 `B2` 中分别定义，而控制流在块 `B3` 汇合，那么就需要在 `B3` 插入 $a = \phi(a1, a2)$ 。函数首先调用 `collectDefBlocks()` 收集每个变量的所有定义块位置，这是 ϕ 插入的基础。随后使用 `hasPhi` 记录哪些块已经为哪些变量插入过 ϕ ，防止重复插入。接下来，函数会遍历所有变量，并采用经典的 Cytron 算法基于支配边界插入 ϕ 。对于某个变量，如果它只在一个块中定义，则不需要 ϕ ；只有当它在两个及以上基本块中定义时，才可能在汇合点产生冲突，因此才进入处理流程。函数将这些定义块加入工作队列 `worklist` 中，不断取出一个定义块 `B`，查看它的支配边界 `DF(B)`。支配边界中的块 `Y` 就是潜在需要插入 ϕ 的位置，因为这些块正是控制流重新汇合的地方。如果块 `Y` 尚未为变量 `v` 插入 ϕ ，就在这里创建一个新的 ϕ 语句。在真正创建 ϕ 时，代码首先通过

lookupTempType() 查找该变量的类型，因为 ϕ 函数必须保留完整类型信息。然后根据该块的所有前驱块，构造 ϕ 的参数列表，每个前驱块都对应一个参数位置，初始值先用占位变量 Temp(0) 填充，后续在 Rename 阶段再由 fillPhiArgFromPred() 真正写入正确版本。接着创建 ϕ 左值变量 phiDst，并初始化它的 def 集合和 use 集合，保证数据流信息完整。之后将 ϕ 插入到基本块的开头位置，但必须放在 LABEL 之后，因为 SSA 要求块结构必须满足“LABEL $\rightarrow$ PHI $\rightarrow$ 普通语句”的顺序。最后，由于插入 ϕ 本身也会形成新的定义点，因此函数会将该块重新加入该变量的定义集合中，并放回工作队列继续传播。这是 Cytron 算法中的关键递归思想，可以确保所有需要 ϕ 的位置都被完整覆盖，而不会遗漏多层控制流汇合点。所有变量处理完成后，再调用 sortPhiPrefix() 对每个基本块中的 ϕ 函数进行统一排序，保证输出规范。

19. renameVariables 函数的作用是完成 SSA 构造中的第二个核心阶段——变量重命名，确保程序真正满足“每个变量只被定义一次”的 SSA 性质。在前面的 placePhi() 中，系统只是把 ϕ 函数插入到了正确的位置，但此时所有变量仍然使用原始的 temp 编号，例如多个赋值仍然可能都写成同一个变量 t101。而 SSA 要求每一次定义都必须产生一个全新的版本，例如 t101_0、t101_1、t101_2。因此这个函数的任务就是沿着支配树遍历整个函数，将所有变量重新编号。函数首先创建 map<int, vector<int>> stacks，它本质上是“变量版本栈”，用于记录每个逻辑变量当前可见的最新版本。例如变量 101 当前若已经定义了两个版本，栈中就会保存 [0,1]，其中栈顶表示当前应被 use 引用的版本。随后将函数原有的最大 temp 编号赋值给 maxTemp，用于追踪新生成的 SSA temp 编号上界。最后从入口基本块 entryBlock 开始调用 renameBlock()，递归沿支配树深度优先遍历整个函数，完成所有 use 的替换和所有 def 的重新编号。
20. cleanupUnusedPhi 函数的作用是在 SSA 转换完成后，删除那些实际上没有被使用的冗余 ϕ 函数。因为 placePhi() 阶段采用的是基于 Dominance Frontier 的标准算法，它的目标是“保证不漏插”，因此往往会插入一些理论上可能需要、但实际上最终没有任何 use 的 ϕ 节点。例如某个变量虽然在控制流汇合点被插入了 ϕ ，但后续程序根本没有再使用这个变量，那么这个 ϕ 就是无意义的冗余节点。保留这些节点虽然不影响程序正确性，但会使 IR 变得臃肿，也会影响后续优化和测试输出。因此该函数首先遍历整个函数中的所有基本块和所有语句，调用 collectUsedTempsStm() 收集所有真正被使用过的 temp 编号，形成一个全局 used 集合。随后再次遍历所有基本块，对其中的 ϕ 语句逐个检查：如果 ϕ 左值定义的变量编号不在 used 集合中，说明这个 ϕ 的结果从未被任何语句引用，那么它就是死代码，可以直接删除。这里通过构造新的 newList 来保留有效语句，删除无用 ϕ 。
21. quad2ssa 函数是整个模块的总入口，也是普通 Quad IR 转换为 SSA 形式的完整驱动函数。它负责组织前面所有步骤，按顺序完成 ϕ 插入、变量重命名以及冗余 ϕ 清理，最终返回完整的 SSA 程序。函数首先检查输入的函数流信息集合 allFuncFlow 是否为空，若为空则说明程序非法，直接返回空指针。随后将所有函数信息转为 vector，并调用 sortFuncFlowForOutput() 对函数顺序进行排序，确保最终输出顺序与课程测试样例完全一致。接着创建新的函数列表 funcs，并初始化整个程序级别的最大 label 编号和最大 temp 编号，用于最终构造新的 QuadProgram。然后函数逐个处理每一个函数 FuncFlowInfo。对于每个函数，首先提取其原始四元式函数声明 funcdecl，以及控制流信息 ControlFlowInfo 和数据流信息 DataFlowInfo。其中 ControlFlowInfo 提供支配树、支配边界、前驱后继关系等 SSA 必需的信息，而 DataFlowInfo 提供活跃变量分析结果，虽然当前实现中主要依赖控制流信息，但接口仍然保留以便后续扩展。接下来依次执行三个核心步骤：首先调用 placePhi()，在控制流汇合点插入 ϕ 函数；然后调用 renameVariables()，沿支配

树递归重命名所有变量，使每次定义都生成新的版本；最后调用 `cleanupUnusedPhi()`，删除没有实际用途的冗余 ϕ 节点。这个顺序是严格固定的，因为必须先有 ϕ ，才能进行重命名；而只有重命名结束后，才能判断哪些 ϕ 真正无用。在完成某个函数的 SSA 转换后，函数会将处理后的 `funcdecl` 加入结果列表中，并同步更新整个程序的最大 label 编号和最大 temp 编号。这样做的原因是，SSA 转换过程中会不断创建新的变量版本和新的 ϕ 节点，因此程序的编号上界一定会发生变化，必须重新维护，否则后续阶段可能出现编号冲突。最后，当所有函数都处理完成后，函数使用新的函数列表以及更新后的全局编号信息，构造并返回一个新的 `QuadProgram` 对象。至此，原始的普通 Quad 四元式程序就被完整转换为了合法、规范、可用于后续优化的 SSA 形式。可以说，`quad2ssa()` 就是整个实验中最核心的总控函数，它把前面所有辅助函数串联起来，真正完成了从普通 IR 到 SSA IR 的全过程。

开发过程和测试结果

我的开发过程的 git graph截图如下（因为push不上去，所以我其实是开发完后在自己的git仓库上又重新提交的）：

Graph		Description	Date	Author	Commit
	 master	feat: 完成了SSA变换的所有逻辑并校正tests输出	25 Apr 2026 14:37	23300240034-chen...	1fcd638a
		begin: 添加了HW7所有文件	25 Apr 2026 14:31	23300240034-chen...	e9820f89

`make run` 之后得到的所有测试的结果，与老师给出的参考测试结果完全一致，出了有些行的临时变量t的编号顺序有前后调整。并且在终端得到的输出片段如下：

```
Block: 100 -> -1
Dominance Frontier:
Block: 100 ->
Dominator Tree:
Block: 100 ->
Liveness information for function: B^f
Block 100:
Statement LABEL Live-in: {t100} Live-out: {t100}
Statement PTR_CALC Live-in: {t100} Live-out: {t100, t102}
Statement STORE Live-in: {t100, t102} Live-out: {t100}
Statement PTR_CALC Live-in: {t100} Live-out: {t100, t103}
Statement STORE Live-in: {t100, t103} Live-out: {t100}
Statement PTR_CALC Live-in: {t100} Live-out: {t100, t104}
Statement STORE Live-in: {t100, t104} Live-out: {t100}
Statement PTR_CALC Live-in: {t100} Live-out: {t100, t105}
Statement LOAD Live-in: {t100, t105} Live-out: {t100, t106}
Statement PTR_CALC Live-in: {t100, t106} Live-out: {t100, t106, t107}
Statement LOAD Live-in: {t100, t106, t107} Live-out: {t100, t106, t108}
Statement MOVE_BINOP Live-in: {t100, t106, t108} Live-out: {t100, t109}
Statement PTR_CALC Live-in: {t100, t109} Live-out: {t109, t110}
Statement LOAD Live-in: {t109, t110} Live-out: {t109, t111}
Statement MOVE_BINOP Live-in: {t109, t111} Live-out: {t112}
Statement RETURN Live-in: {t112} Live-out: {}
Control Flow Information for function: B^f
Predecessors:
Block: 100 <-
Successors:
Block: 100 ->
Dominators:
Block: 100 <- 100
Immediate Dominators:
Block: 100 -> -1
Dominance Frontier:
Block: 100 ->
Dominator Tree:
Block: 100 ->
Converting Quad to Quad-SSA
Done converting Quad to Quad-SSA
Writing Quad-SSA to: quadtest9.4-ssa.quad
Done writing Quad-SSA (text) to: quadtest9.4-ssa.quad
-----Done---
```

参考资料

- 虎书中文版，“静态单赋值形式”，“数据流分析”，“循环优化”部分
- 大模型：DeepSeek/Cursor，协助理解虎书内容和修改代码中遇到的顺序调整问题。